Page Replacement Algorithm Simulator

Project Report

Operating Systems Lab | Virtual Memory / Page Replacement

Name: KUMAR RISHI **Enrollment No.:** BTE25DTA100006

Date: 25 October 2026

Language: Python 3 (Jupyter Notebook) **Notebook:** Page_Replacement_Simulator.ipynb

1. Introduction

In a demand-paged system, a process runs with only some of its pages in main memory. When it references a page that is not in a frame, a page fault occurs. If every frame is full, the operating system selects a victim page to replace. A reference to a page already in memory is a page hit. The replacement policy affects the number of faults and therefore memory performance.

Objectives

• Implement FIFO, LRU and Optimal page replacement.

• Accept a page reference string and frame count at runtime, with input validation.

• Display step-by-step frame contents, Hit/Fault status and replaced page.

• Calculate page faults, page hits, hit ratio and fault ratio.

• Compare all algorithms and investigate Belady's anomaly.

2. Algorithms

Algorithm	Victim selection rule	Notes
FIFO	Page loaded earliest	Simple queue; can exhibit Belady's anomaly.
LRU	Page least recently used in the past	Tracks last-use information; often benefits from locality.
Optimal	Page whose next use is farthest away, Or never again	Minimum-fault benchmark; future references are unavailable in real OS operation

2.1 FIFO

FIFO maintains pages in arrival order. When a fault occurs with full frames, the oldest resident page is replaced. It is easy to implement but does not consider recent use or future need.

2.2 LRU

LRU replaces the page that has not been used for the longest time in the past. The simulator records the last-use index for each resident page. Real implementations require recency bookkeeping or hardware support.

2.3 Optimal

Optimal replaces the resident page whose next use is farthest in the future, or a page that will not be used again. If multiple pages are never used again, the simulator replaces the one appearing first in the frame list. This algorithm is used as a benchmark.

3. Implementation

The notebook uses Python and pandas. The functions parse_reference_string(), simulate(), fifo(), lru(), optimal(), print_trace() and compare_algorithms() handle input parsing, algorithm simulation, trace display and comparison. All frames start empty and fill in slot order. A replaced page occupies the same frame slot. Each trace records step, referenced page, frame contents, H/F result and replaced page.

Input validation

Input issue	Message / behaviour
Empty reference string	Enter at least one page number.
Non-numeric page	Page references must be whole numbers separated by commas or spaces.
Negative page	Page numbers must be non-negative integers.
Non-integer frame count	Number of frames must be a whole number.
Frame count below 1	Number of frames must be at least 1.

4. Sample Input and Output

Input reference string: 1 2 3 1 4 2 5; number of frames: 3.

Algorithm	Page faults	Page hits	Hit ratio	Fault ratio
FIFO	5	2	28.57%	71.43%
LRU	6	1	14.29%	85.71%
Optimal	5	2	28.57%	71.43%

Example trace: Optimal

Step	Page	Frame contents	H/F	Replaced
1	1	[1]	F	-
2	2	[1, 2]	F	-
3	3	[1, 2, 3]	F	-
4	1	[1, 2, 3]	H	-
5	4	[4, 2, 3]	F	1
6	2	[4, 2, 3]	H	-
7	5	[5, 2, 3]	F	4

Execution screenshots are supplied in the screenshots folder. They show the sample algorithm traces, comparison table, and validation examples.

5. Part B — Manual Tracing

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2. Frames: 3. F = page fault; H = page hit; '-' means an empty frame.

FIFO

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2	-	0	0	0	0	3	3	3	2	2	2	2	2
Frame 3	-	-	1	1	1	1	0	0	0	3	3	3	3
F/H	F	F	F	F	H	F	F	F	F	F	F	H	H

Page faults: 10; page hits: 3; hit ratio: 23.08%; fault ratio: 76.92%.

LRU

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0
Frame 2	-	0	0	0	0	0	0	0	0	3	3	3	3
Frame 3	-	-	1	1	1	3	3	3	2	2	2	2	2
F/H	F	F	F	F	H	F	H	F	F	F	F	H	H

Page faults: 9; page hits: 4; hit ratio: 30.77%; fault ratio: 69.23%.

Optimal

Ref	7	0	1	2	0	3	0	4	2	3	0	3	2
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2
Ref	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**	**2**
Frame 2	-	0	0	0	0	0	0	4	4	4	0	0	0
Frame 3	-	-	1	1	1	3	3	3	3	3	3	3	3
F/H	F	F	F	F	H	F	H	F	H	H	F	H	H

Page faults: 7; page hits: 6; hit ratio: 46.15%; fault ratio: 53.85%.

Part B summary

Algorithm	Page faults	Page hits	Hit ratio
FIFO	10	3	23.08%
LRU	9	4	30.77%
Optimal	7	6	46.15%

6. Part C — Experiments and Analysis

6.1 Experiment 1: Belady's anomaly

Reference string: 1 2 3 4 1 2 5 1 2 3 4 5.

Algorithm	Faults (3 frames)	Faults (4 frames)
FIFO	9	10
LRU	10	8
Optimal	7	6

Q1. Does any algorithm produce more faults with more frames? Yes. FIFO increases from 9 faults with 3 frames to 10 faults with 4 frames. This is Belady's anomaly.

Q2. Why do LRU and Optimal not show this behaviour? They have the stack property: the set of pages resident with n frames is a subset of those resident with n+1 frames. An extra frame therefore cannot increase their faults. FIFO does not have this property.

6.2 Experiment 2: Custom inputs (4 frames)

Case	Reference string	Frames	FIFO faults	LRU faults	Optimal faults
LRU better	1 1 1 4 3 5 5 4 3 2 2 1 4 2 4 3 2	4	8	6	5
FIFO better/equal	5 3 5 2 3 1 1 2 3 5 5 2 1 3 2 3 4 1	4	5	5	5

The first custom case is intended to demonstrate LRU performing better than FIFO; the second demonstrates FIFO being equal to LRU, which meets the assignment's "better than or equal to" condition. The exact results are calculated by the notebook and included in the table.

6.3 Short questions

1. **Why can Optimal not be implemented in a real OS?** It needs future page references, which cannot be known in advance. It is used as a benchmark.

2. **What extra hardware or bookkeeping does LRU need compared with FIFO?** LRU must track each page's last-use time, using timestamps/counters or a stack/list updated on accesses. FIFO only maintains arrival order.

3. **If frames equal the number of distinct pages, how many faults will every algorithm produce?** One fault for each distinct page. Once loaded, no page needs to be evicted, so all later references are hits.

7. Comparison of Results

Test input	Frames	FIFO faults	LRU faults	Optimal faults
Sample string	3	5	6	5
Part B string	3	10	9	7
Belady string	3	9	10	7
Belady string	4	10	8	6
Custom LRU case	4	8	6	5
Custom tie case	4	5	5	5

- Optimal provides the minimum number of faults for a given input and frame count.
- LRU often performs better than FIFO when recently used pages are reused, but it is not always better.
- FIFO can show Belady's anomaly: increasing frames can increase faults.
- FIFO is simple to implement; LRU requires recency tracking; Optimal is a benchmark because future references areunavailable.

8. Conclusion

The simulator implements FIFO, LRU and Optimal page replacement with validated user input, step-by-step traces and a comparison of page faults, hits, hit ratio and fault ratio. The experiments illustrate the strengths and limitations of each algorithm and demonstrate Belady's anomaly in FIFO. The notebook can be run with new reference strings to explore how page replacement affects memory performance.